CENG-322 Mobile Application Development - Deliverable 2

Team Name: IoT Innovators

Project Name: Smart Farm

Team Number: 8

Student's names and IDs:

- Chau Vi Cuong Nguyen – N01634611
- Hoang Tran – N01613506
- Tuan Anh Pham – N01238459
- Truong Anh Nguyen - N01639800

Table of Contents

Member Info and Participation 2
Project Scope and goals that are targeted in this course 3
GitHub Repo link and strategy 3
Stories and Task breakdown 4
List of items you included in the DoD. 6
Business Model Canvas 7
1. Customer Relationships 8
2. Value Propositions 8
3. Channels 8
4. Customer Relationships 9
5. Revenue Streams 9
6. Key Activities 10
7. Key Resources 10
8. Key Partnerships 11
9. Cost Structure 11
Gantt chart 12
Details and type of DB 13
Work Progress 13
Meetings Note 14
Design Principle 14

Member Info and Participation

Name	Student ID	GitHub ID	Signature	Effort
Chau Vi Cuong Nguyen	N01634611	ChauViCuongNguyen4611	CVCN	100
Hoang Tran	N01613506	HoangTranTran3506	HT	100
Tuan Anh Pham	N01238459	TuanAnhPham8459	TA	100
Truong An Nguyen	N01639800	TruongAnNguyen9800	TAN	90

Project Scope and goals that are targeted in this course

The Smart Farm IoT application aims to create a comprehensive mobile solution for modern farming management. The final vision is to provide farmers with real-time monitoring and control of IoT-enabled farm equipment, including temperature monitors, automated irrigation systems, and crop health trackers.

GitHub Repo link and strategy

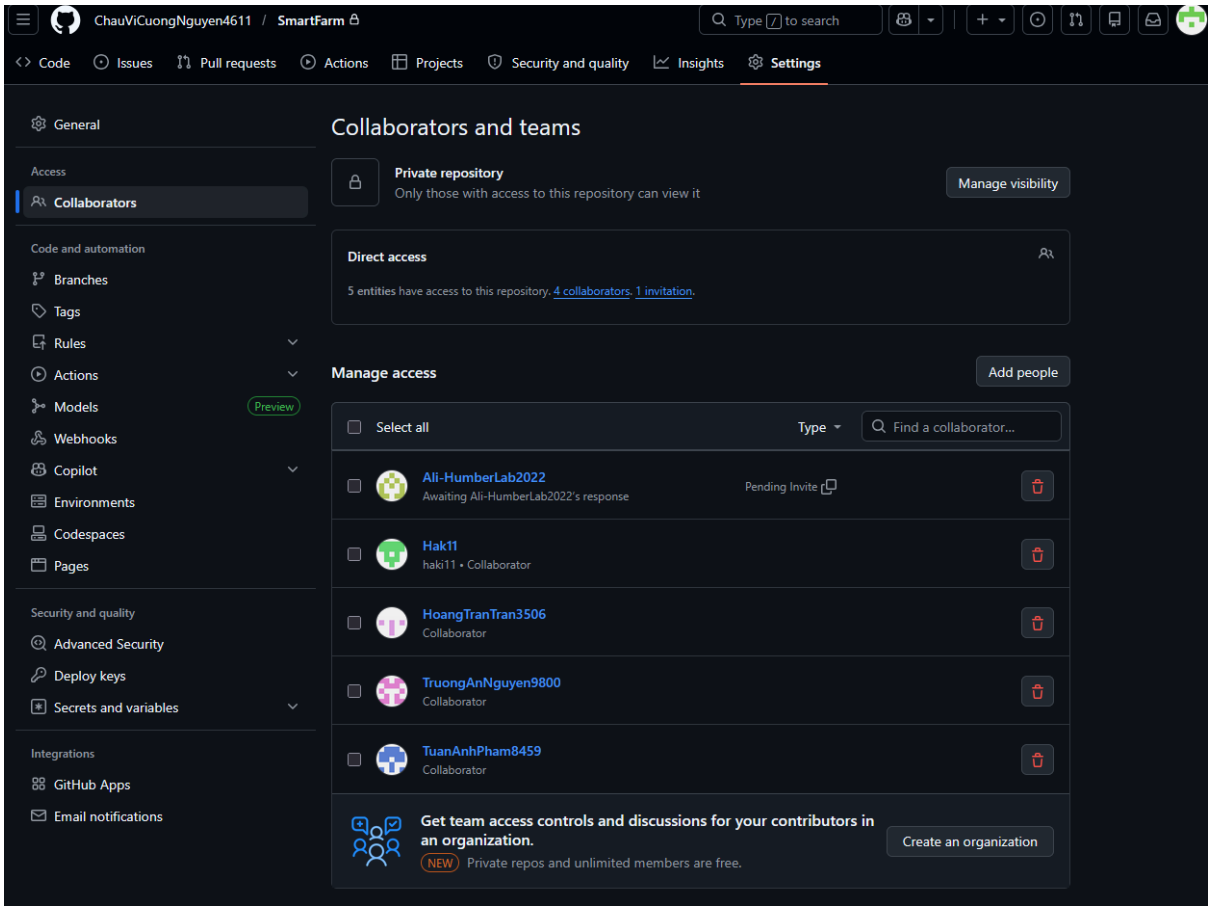
https://github.com/ChauViCuongNguyen4611/SmartFarm

Because of a small team with frequent deployment cycle, the team will focus on utilizing GitHub Flow for Branching Strategy.

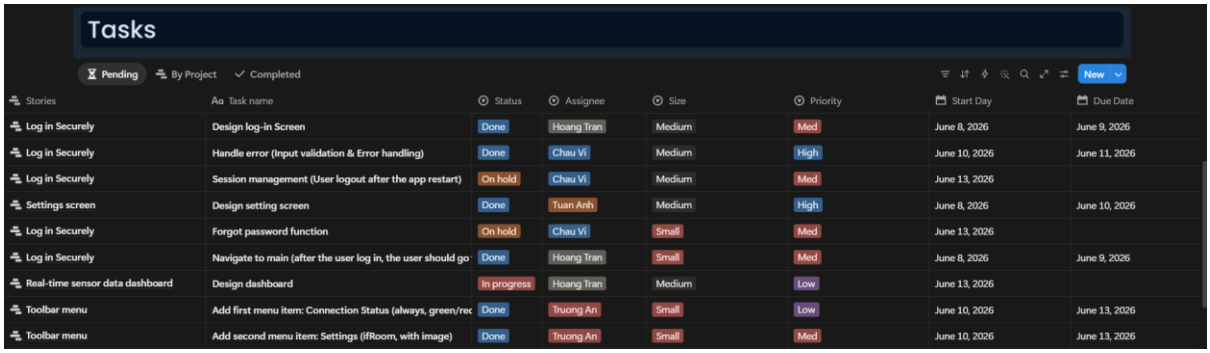
The core principles focus on keeping main branch always in a deployable state:

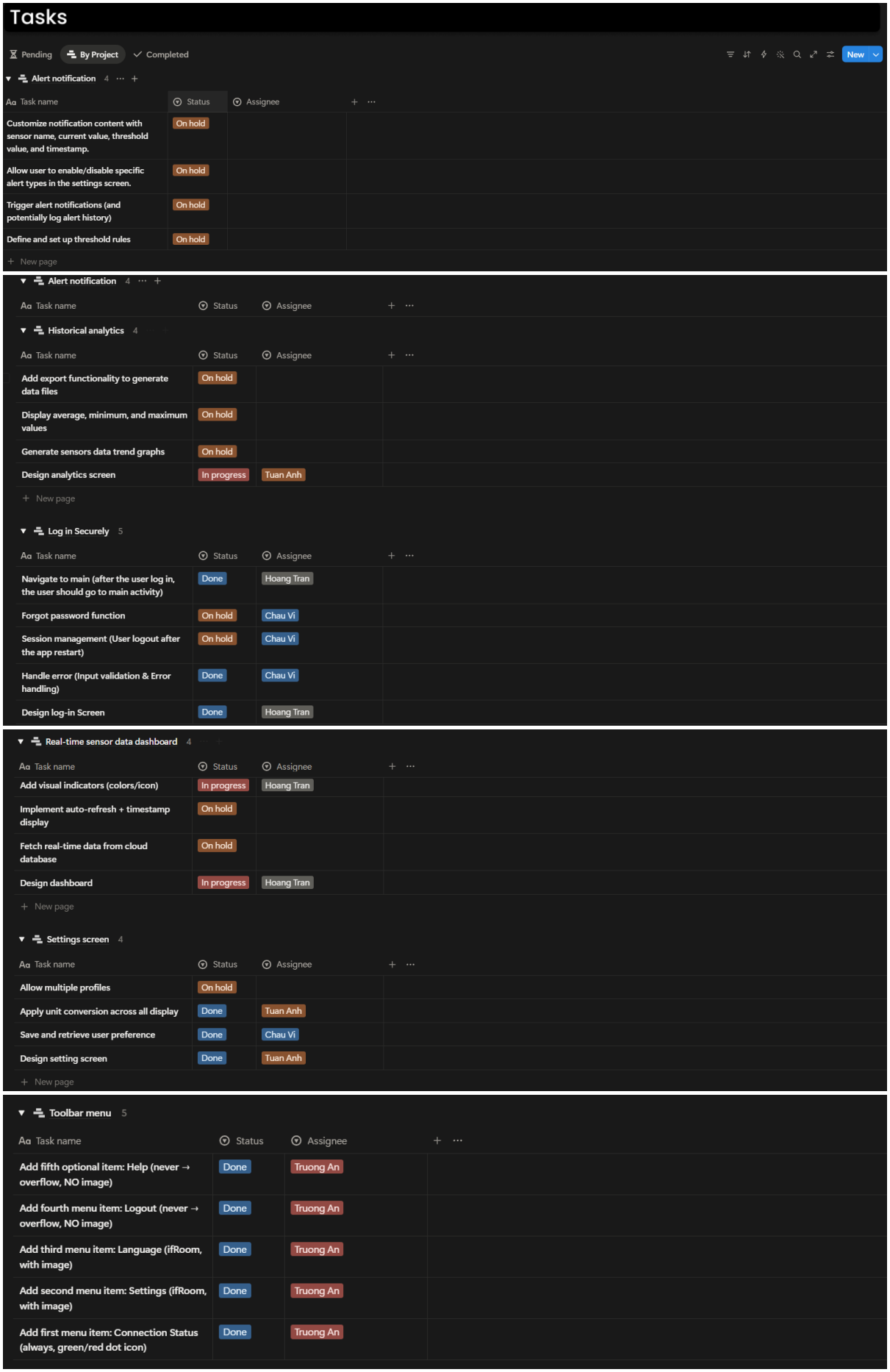
- Avoid committing directly to main: Create a branch with a descriptive name to commit your work.
- Keep branches short-lived: Keep branches focusing on single features.
- Use Pull Request to propose changes: a collaboration to review code and discuss about requested branch will be held
- Merge the PR once approved and deploy immediately

Screenshot showing you have invited your hardware prof to the repo



Stories and Task breakdown





Tasks

Pending | By Project | Completed | New

Alert notification 4

Task name	Status	Assignee
Customize notification content with sensor name, current value, threshold value, and timestamp.	On hold	
Allow user to enable/disable specific alert types in the settings screen.	On hold	
Trigger alert notifications (and potentially log alert history)	On hold	
Define and set up threshold rules	On hold	

New page

Alert notification 4

Task name	Status	Assignee

Historical analytics 4

Task name	Status	Assignee
Add export functionality to generate data files	On hold	
Display average, minimum, and maximum values	On hold	
Generate sensors data trend graphs	On hold	
Design analytics screen	In progress	Tuan Anh

New page

Log in Securely 5

Task name	Status	Assignee
Navigate to main (after the user log in, the user should go to main activity)	Done	Hoang Tran
Forgot password function	On hold	Chau Vi
Session management (User logout after the app restart)	On hold	Chau Vi
Handle error (Input validation & Error handling)	Done	Chau Vi
Design log-in Screen	Done	Hoang Tran

Real-time sensor data dashboard 4

Task name	Status	Assignee
Add visual indicators (colors/icon)	In progress	Hoang Tran
Implement auto-refresh + timestamp display	On hold	
Fetch real-time data from cloud database	On hold	
Design dashboard	In progress	Hoang Tran

New page

Settings screen 4

Task name	Status	Assignee
Allow multiple profiles	On hold	
Apply unit conversion across all display	Done	Tuan Anh
Save and retrieve user preference	Done	Chau Vi
Design setting screen	Done	Tuan Anh

New page

Toolbar menu 5

Task name	Status	Assignee
Add fifth optional item: Help (never → overflow, NO image)	Done	Truong An
Add fourth menu item: Logout (never → overflow, NO image)	Done	Truong An
Add third menu item: Language (ifRoom, with image)	Done	Truong An
Add second menu item: Settings (ifRoom, with image)	Done	Truong An
Add first menu item: Connection Status (always, green/red dot icon)	Done	Truong An

List of items you included in the DoD.

Code & Implementation

- No compilation errors or warnings introduced
- Code is written and functional according to the task description
- No hardcoded strings, colors, or dimensions (uses resources files)
- Code follows Java naming conventions and is properly indented
- Class/method complexity is reasonable (single responsibility principle)

Testing & Verification

- Task works on API 31-36
- Task works in both English and French languages
- Edge cases handled (no network, empty data, invalid input)
- No crashes during normal usage flow

User Experience (UI-related tasks only)

- Task works in both portrait and landscape orientations (if UI-related)
- All images have minimum 3 resolutions (mdpi, hdpi, xhdpi)
- Proper elevation and Material Design guidelines followed
- Visual feedback provided for user actions (toast/snackbar/animation)

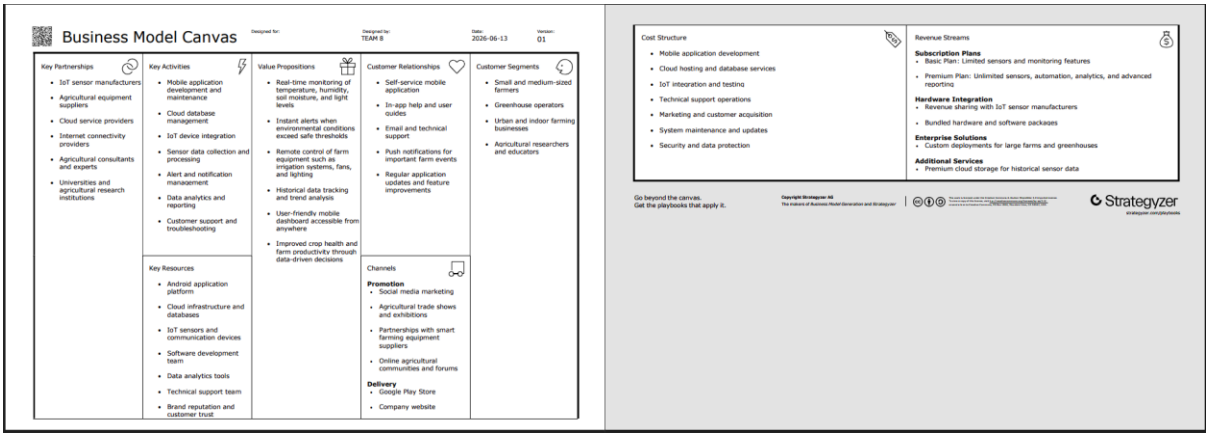
Permissions & Navigation (if applicable)

- Runtime permissions requested properly with explanation if denied
- Toast displayed when permission granted or denied
- Navigation between screens works without issues

Git & Documentation

- Code committed with meaningful commit message
- Code reviewed and merged into main branch
- No merge conflicts remaining

Business Model Canvas



Business Model Canvas

Designed for:

Designed by: TEAM 8

Date: 2026-06-13

Version: 01

Key Partnerships
- IoT sensor manufacturers
- Agricultural equipment suppliers
- Cloud service providers
- Internet connectivity providers
- Agricultural consultants and experts
- Universities and agricultural research institutions

Key Activities
- Mobile application development and maintenance
- Cloud database management
- IoT device integration
- Sensor data collection and processing
- Alert and notification management
- Data analytics and reporting
- Customer support and troubleshooting

Key Resources
- Android application platform
- Cloud infrastructure and databases
- IoT sensors and communication devices
- Software development team
- Data analytics tools
- Technical support team
- Brand reputation and customer trust

Value Propositions
- Real-time monitoring of temperature, humidity, soil moisture, and light levels
- Instant alerts when environmental conditions exceed safe thresholds
- Remote control of farm equipment such as irrigation systems, fans, and lighting
- Historical data tracking and trend analysis
- User-friendly mobile dashboard accessible from anywhere
- Improved crop health and farm productivity through data-driven decisions

Customer Relationships
- Self-service mobile application
- In-app help and user guides
- Email and technical support
- Push notifications for important farm events
- Regular application updates and feature improvements

Channels

Promotion
- Social media marketing
- Agricultural trade shows and exhibitions
- Partnerships with smart farming equipment suppliers
- Online agricultural communities and forums

Delivery
- Google Play Store
- Company website

Customer Segments
- Small and medium-sized farmers
- Greenhouse operators
- Urban and indoor farming businesses
- Agricultural researchers and educators

Cost Structure
- Mobile application development
- Cloud hosting and database services
- IoT integration and testing
- Technical support operations
- Marketing and customer acquisition
- System maintenance and updates
- Security and data protection

Revenue Streams

Subscription Plans
- Basic Plan: Limited sensors and monitoring features
- Premium Plan: Unlimited sensors, automation, analytics, and advanced reporting

Hardware Integration
- Revenue sharing with IoT sensor manufacturers
- Bundled hardware and software packages

Enterprise Solutions
- Custom deployments for large farms and greenhouses

Additional Services
- Premium cloud storage for historical sensor data

1. Customer Relationships

Customer Segments identify the groups of people or organizations that will use or pay for our product.

For the Smart Farm App:

- Small-scale farmers
- Greenhouse operators
- Urban farmers
- Agricultural researchers

What they think, see, feel, and do:

- **Think:** "I need an easier way to monitor my farm."
- **See:** Environmental conditions changing throughout the day.
- **Feel:** Concerned about crop damage and wasted resources.
- **Do:** Check sensor readings, adjust equipment, and monitor farm performance.

Understanding our customers helps us design features that solve their actual problems.

2. Value Propositions

The Value Proposition describes the problem we solve and why customers would choose our product over alternatives.

For the Smart Farm App:

- Real-time farm monitoring
- Remote control of irrigation and environmental systems
- Instant notifications for abnormal conditions
- Historical data analysis
- Improved crop productivity
- Reduced labor and operational costs

Why customers buy or use it:

- Saves time
- Prevents crop loss
- Reduces water and energy waste
- Makes farming decisions easier

3. Channels

Channels are how customers discover, purchase, and use our product.

For the Smart Farm App:

Promotion

- Social media
- Agricultural websites
- Farming exhibitions
- Online advertisements

Distribution

- Android app through Google Play Store
- Direct installation by partners
- Company website

4. Customer Relationships

Customer Relationships describe how we interact with users before, during, and after they use our product.

For the Smart Farm App:

- User registration and onboarding
- In-app tutorials
- Technical support
- Push notifications
- Regular updates
- Feedback collection

Customer Journey Example

1. User downloads app
2. Connects sensors
3. Receives alerts
4. Reviews historical data
5. Continues subscription

Strong customer relationships improve retention and satisfaction.

5. Revenue Streams

Revenue Streams describe how the business earns money.

For the Smart Farm App:

Subscription Plans

- Basic plan: free or low-cost

- Premium plan: advanced analytics and automation

Hardware Sales

- Smart sensors
- Farm monitoring kits

Enterprise Contracts

- Large-scale farm deployments

Additional Services

- Premium cloud storage

Example

A farmer pays $9.99/month for premium monitoring and automation features.

6. Key Activities

Key Activities are the most important tasks the company performs to deliver value.

For the Smart Farm App:

- Developing the Android application
- Managing cloud databases
- Processing sensor data
- Sending notifications
- Maintaining servers
- Providing customer support
- Integrating IoT devices

Example

When a sensor detects high temperature, the system processes the data and sends an alert.

These activities directly create the value customers pay for.

7. Key Resources

What it means:

Key Resources are the assets needed to operate the business.

For the Smart Farm App:

Physical Resources

- Sensors
- Testing equipment

Technology Resources

- Mobile application
- Cloud servers
- Databases

Human Resources

- Software developers
- Support staff
- Agricultural experts

Intellectual Resources

- Software code
- Algorithms
- Brand reputation

Why it's important:

Without these resources, the business cannot deliver its services.

8. Key Partnerships

Key Partnerships are organizations that help our business operate successfully.

For the Smart Farm App:

- Sensor manufacturers
- Cloud service providers
- Agricultural equipment companies
- Internet providers
- Research institutions

Example

Instead of building sensors ourselves, we partner with a sensor manufacturer.
Partnerships reduce costs, risks, and development time.

9. Cost Structure

Cost Structure identifies the major expenses required to run the business.
Understanding costs helps ensure the business remains profitable.

For the Smart Farm App:

Development Costs

- Software development

- Testing

Infrastructure Costs

- Cloud hosting
- Databases
- Data storage

Operational Costs

- Technical support
- Maintenance
- Security monitoring

Marketing Costs

- Advertising
- Promotions
- Trade shows

Relationship to Revenue

- More users → higher cloud costs
- More premium subscribers → higher revenue
- Better features → higher development costs but potentially more customers

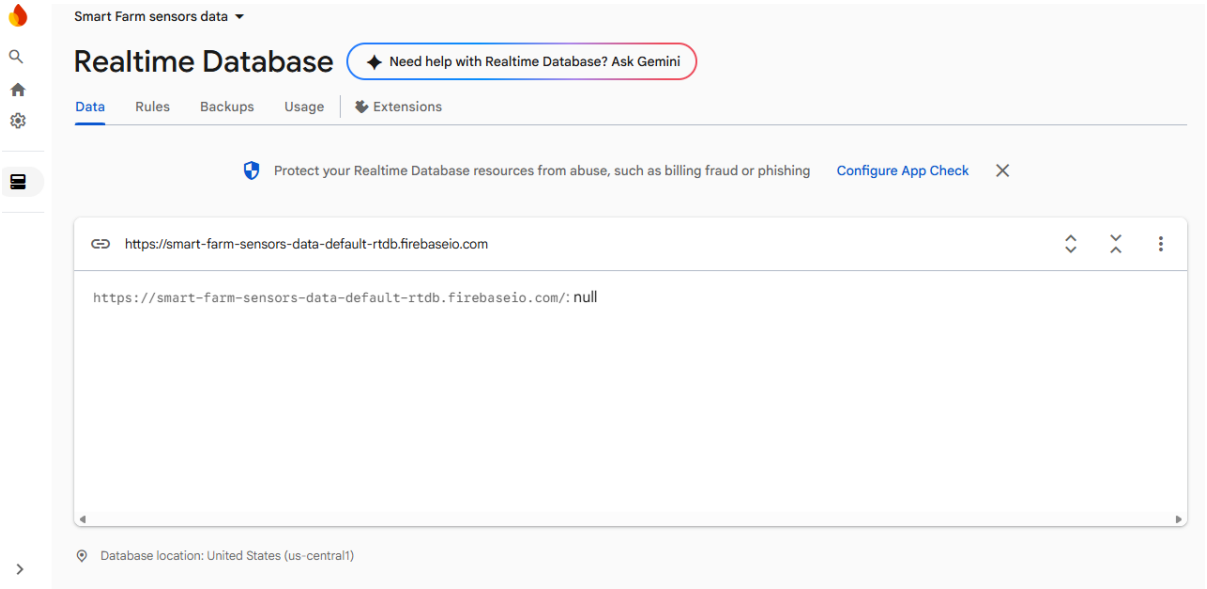
Gantt chart

Grid view | Hide fields | Filter | Group | Sort

	Task	Start	End	Phase	Assignee
1	Task			Phase	Assignee
2	Deliverable 0: Team Format...	2026-05-18	2026-05-24	Planning	All
3	Define App Architecture & ...	2026-05-18	2026-05-24	Planning	Software Lead
4	Setup GitHub Repository &...	2026-05-25	2026-05-31	Deliverable 1	Scrum Master
5	Android App Initialization ...	2026-05-25	2026-06-07	Deliverable 1	App Devs
6	Design Day/Night Splash S...	2026-06-01	2026-06-07	Deliverable 1	App Devs
7	Create Firebase Realtime D...	2026-06-08	2026-06-14	Deliverable 2	Backend Dev
8	Set up Scrum Dashboard (T...	2026-06-08	2026-06-14	Deliverable 2	Scrum Master
9	Design 5 Main App Screens...	2026-06-08	2026-06-21	Deliverable 2	App Devs
10	Implement Settings Screen ...	2026-06-15	2026-06-21	Deliverable 2	App Devs
11	Create Gantt Chart & Busin...	2026-06-15	2026-06-21	Deliverable 2	Team Lead
12	Write Script to Push Mock ...	2026-06-22	2026-06-28	Deliverable 3 (Forecast)	Backend Dev

12	Write Script to Push Mock ...	2026-06-22	2026-06-28	Deliverable 3 (Forecast)	Backend Dev
13	Fetch Real-Time Firebase D...	2026-06-29	2026-07-05	Deliverable 3 (Forecast)	App Devs
14	Develop UI for Automated ...	2026-06-29	2026-07-05	Deliverable 3 (Forecast)	App Devs
15	Implement App Logic for T...	2026-07-06	2026-07-12	Deliverable 4 (Forecast)	App Devs
	Implement Push Notificatio...	2026-07-10	2026-07-19	Deliverable 4 (Forecast)	App Devs
17	Fully Integrate English/Fren...	2026-07-10	2026-07-19	Deliverable 4 (Forecast)	All
18	End-to-End Software Testin...	2026-07-20	2026-07-26	Deliverable 5 (Forecast)	All
19	Refine UI/UX & Code Clean...	2026-07-25	2026-08-02	Deliverable 5 (Forecast)	App Devs
20	Prepare Final Presentation ...	2026-08-03	2026-08-16	Final Presentation	All

Details and type of DB



We selected Firebase Realtime Database. We plan to use it as the central cloud bridge between our hardware and software. Our greenhouse sensors will push live temperature and moisture data to this database, and our Android app will continuously read from it to display the live dashboard to the user.

Work Progress

Coding work progress

What additional features/functionality added since deliverable 1.

- Add Bottom Navigation to main activity.
- Add New Activity for Log-in feature. App will first launch into Log-in screen.
- Add the Toolbar menu items.
- Rewrite Settings code to use Preference Fragment
- Write function for portrait lock, export import settings in Settings.
- Ask for notification permission to prepare for the alert system.
- Demonstrate the use of style, colors, and strings. Avoid hard-coding in any case.

Meetings Note

A Table records the daily stand-ups outcome,

Date & Time	Attendees	Meeting Outcomes & Decisions	Action Items for Next Meeting
June 2, 2026 2:05 PM - 2:20 PM	Chau Vi Cuong Nguyen Hoang Tran Truong An Nguyen Tuan Anh Pham	- Review previous stories to meet new feedback and outcome - Determined which initial features to focus on.	- Rewrite tasks based on the agreed-upon stories. - Adding missing navigation.
June 8, 2026 5:00 PM - 5:30 PM	Chau Vi Cuong Nguyen Hoang Tran Truong An Nguyen Tuan Anh Pham	- Reviewed current progress. - Re-assigned tasks: Tuan Anh Pham and Hoang Tran handling UI/UX; Truong An Nguyen and Chau Vi Cuong Nguyen focusing on app functions. - Decided on app themes (light/dark mode colors). - Decided on options for the action bar menu.	- Fixing critical bug causing crashes on log in screen - Focus on finishing initial layout
June 12, 2026 3:00 PM - 3:30 PM	Chau Vi Cuong Nguyen Hoang Tran Truong An Nguyen Tuan Anh Pham	- Reviewed current progress. - Assign documentation works. - Identify error causing unable to load on Tran's emulator	- Review individual tasks and prepare to work on the PowerPoint presentation together.

Design Principle

We choose **Single Responsibility Principle (SRP)** as our design principle because in Android development, this means each class should handle one specific responsibility – not multiple unrelated tasks.

Why We Chose SRP for Smart Farm

Benefit	Explanation
Separation of concerns	UI logic is separate from validation logic
Easier testing	LoginValidator can be tested without an Activity context
Reusability	LoginValidator can be used by other Activities (e.g., SignUp)
Maintainability	Changes to validation rules don't affect UI code
Follows course guideline	"Have your Java classes focus on certain functionality. Do not make it too big, and hard to maintain"

This separation means:

If validation rules change → edit only LoginValidator

If UI design changes → edit only LoginActivity

If both change → no conflicts

Code Example 1: LoginActivity.java (UI Responsibility Only)

```
/**
 * This class is ONLY responsible for:
 * 1. Initializing UI views
 * 2. Setting up click listeners
 * 3. Display validation errors from LoginValidator
 * 4. Navigating to the next screen
 *
 * If the UI design changes, we modify ONLY this class.
 * If validation rules change, LoginValidator handles that.
 */
public class LoginActivity extends BaseActivity {
```

```
    // UI components - all related to displaying the login screen
    private TextInputEditText etEmail, etPassword;
    private TextInputLayout tilEmail, tilPassword;
    private MaterialButton btnLogin, btnSignup;
    private TextView tvForgotPassword;

    // Delegate validation to a separate class (SRP)
    private LoginValidator loginValidator;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        // Create validator instance (separate responsibility)
        loginValidator = new LoginValidator(this);

        initializeViews();      // Only UI initialization
        setupClickListeners();  // Only event handling
    }

    /**
     * Initializes all UI views.

     */
    private void initializeViews() {
        tilEmail = findViewById(R.id.til_email);
        tilPassword = findViewById(R.id.til_password);
        etEmail = findViewById(R.id.et_email);
        etPassword = findViewById(R.id.et_password);
        btnLogin = findViewById(R.id.btn_login);
        btnSignup = findViewById(R.id.btn_signup);
        tvForgotPassword = findViewById(R.id.tv_forgot_password);
    }

    /**
     * Sets up click listeners for buttons.
     * SRP: This method ONLY registers listeners.
     * The actual logic is in separate methods.
     */
    private void setupClickListeners() {
        btnLogin.setOnClickListener(v -> validateAndLogin());
        btnSignup.setOnClickListener(v -> navigateToSignup());
```

```
        tvForgotPassword.setOnClickListener(v -> navigateToForgotPassword());
    }

    /**
     * Validates credentials using LoginValidator.
     * Then either shows errors or proceeds with login.
     */
    private void validateAndLogin() {
        // Get user input
        String email = etEmail.getText().toString().trim();
        String password = etPassword.getText().toString().trim();

        // Delegate validation to LoginValidator (separate class, separate
responsibility)
        LoginValidator.ValidationResult result =
loginValidator.validateCredentials(email, password);

        // Handle validation result (UI responsibility only)
        if (!result.isValid()) {
            tilEmail.setError(result.getEmailError());
            tilPassword.setError(result.getPasswordError());
            return;
        }

        // Clear errors and proceed
        tilEmail.setError(null);
        tilPassword.setError(null);
        performLogin(email, password);
    }

     /**
     * Navigation methods - UI responsibility only.
     * Actual logic will be in separate Activities.
     */
    private void navigateToSignup() {
        Toast.makeText(this, R.string.signup_coming_soon, Toast.LENGTH_SHORT).show();
        // TODO: Navigate to SignUpActivity (separate class)
    }

    private void navigateToForgotPassword() {
        Toast.makeText(this, R.string.forgot_password_coming_soon,
Toast.LENGTH_SHORT).show();
        // TODO: Navigate to ForgotPasswordActivity (separate class)
```

```
    }
}
```

Code Example 2: LoginValidator.java (Validation Responsibility Only)

```
/** CENG-322-0CA
 * Chau Vi Cuong Nguyen  N01634611
 * Hoang Tran           N01613506
 * Truong An Nguyen     N01639800
 * Tuan Anh Pham        N01238459
 **/

package ca.iot.innovators.smart.farm;

import android.content.Context;
import android.text.TextUtils;

/**
 * DESIGN PRINCIPLE: Single Responsibility Principle (SRP)
 *
 * This class has ONE and ONLY ONE responsibility:
 * "Validate user login credentials (email and password)."
 */
public class LoginValidator {

    private final Context context;

    public LoginValidator(Context context) {
        this.context = context;
    }

    /**
     * Inner class to hold validation results.
     * This class only exists to return multiple values from validation.
     * It doesn't perform any logic itself.
     */
    public static class ValidationResult {
        private final boolean isValid;
        private final String emailError;
        private final String passwordError;

        public ValidationResult(boolean isValid, String emailError, String
```

```
passwordError) {
            this.isValid = isValid;
            this.emailError = emailError;
            this.passwordError = passwordError;
        }

        public boolean isValid() { return isValid; }
        public String getEmailError() { return emailError; }
        public String getPasswordError() { return passwordError; }
    }

    /**
     * Validates email format.
     * This method ONLY checks if email is valid.
     * Returns error message or null.
     *
     * Validation rules:
     * 1. Email cannot be empty
     * 2. Email must match standard email pattern
     */
    private String validateEmail(String email) {
        if (TextUtils.isEmpty(email)) {
            return context.getString(R.string.email_required);
        } else if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
            return context.getString(R.string.email_invalid);
        }
        return null;  // No error - email is valid
    }

    /**
     * Validates password format.
     * This method ONLY checks if password is valid.
     * Returns error message or null.
     *
     * Validation rules:
     * 1. Password cannot be empty
     * 2. Password must be at least 6 characters
     */
    private String validatePassword(String password) {
        if (TextUtils.isEmpty(password)) {
            return context.getString(R.string.password_required);
       } else if (password.length() < 6) {
            return context.getString(R.string.password_too_short);
        }
```

```
        return null; // No error - password is valid
    }

    /**
     * Main validation method.
     * This method ONLY combines email and password validation.
     * It doesn't know about UI, databases, or navigation.
     */
    public ValidationResult validateCredentials(String email, String password) {
        String emailError = validateEmail(email);
        String passwordError = validatePassword(password);
        boolean isValid = (emailError == null && passwordError == null);

        return new ValidationResult(isValid, emailError, passwordError);
    }
}
```